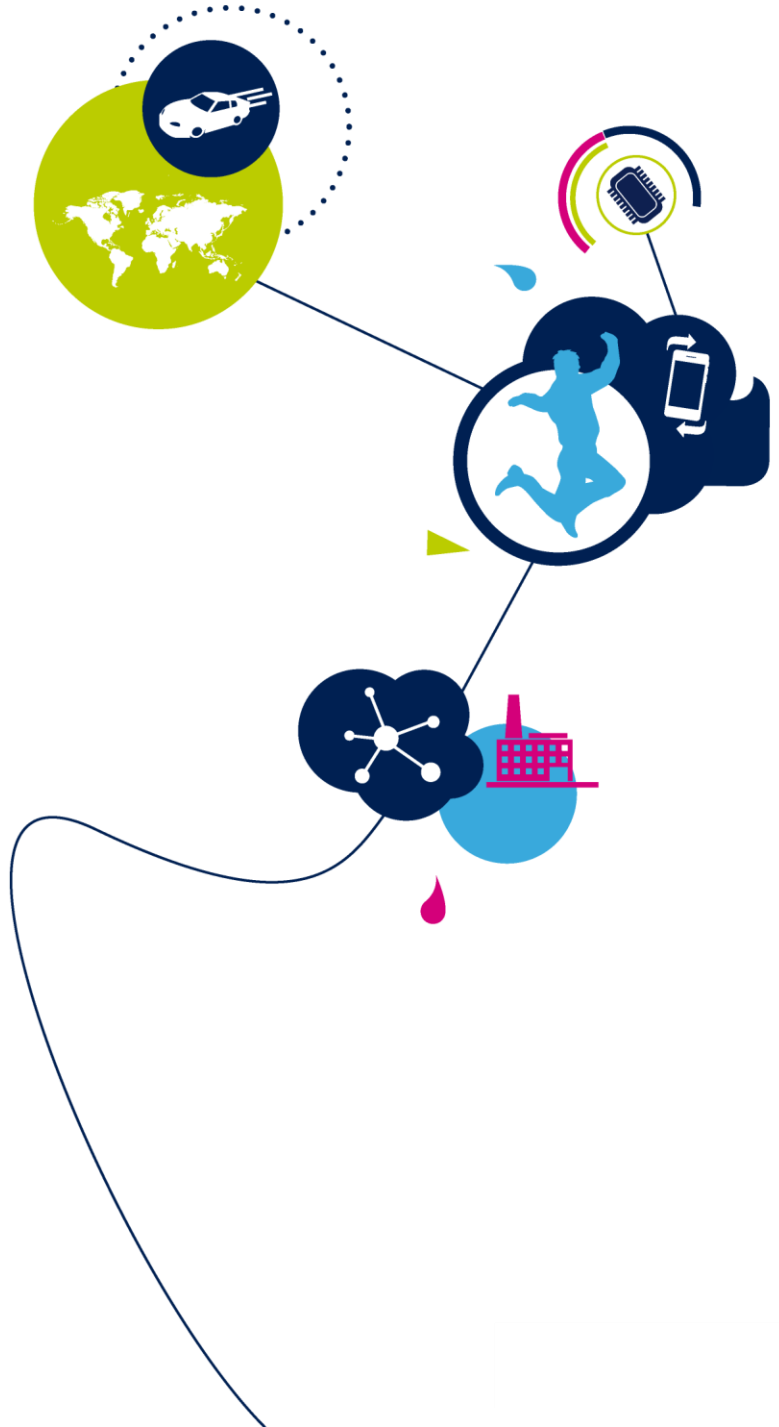
External QSPI loader

T.O.M.A.S. team



life.augmented

Info

- This presentation helps you to integrate QSPI memory to your embedded system
- Presentation is split in two parts
  - Theoretical
  - Practice





Content

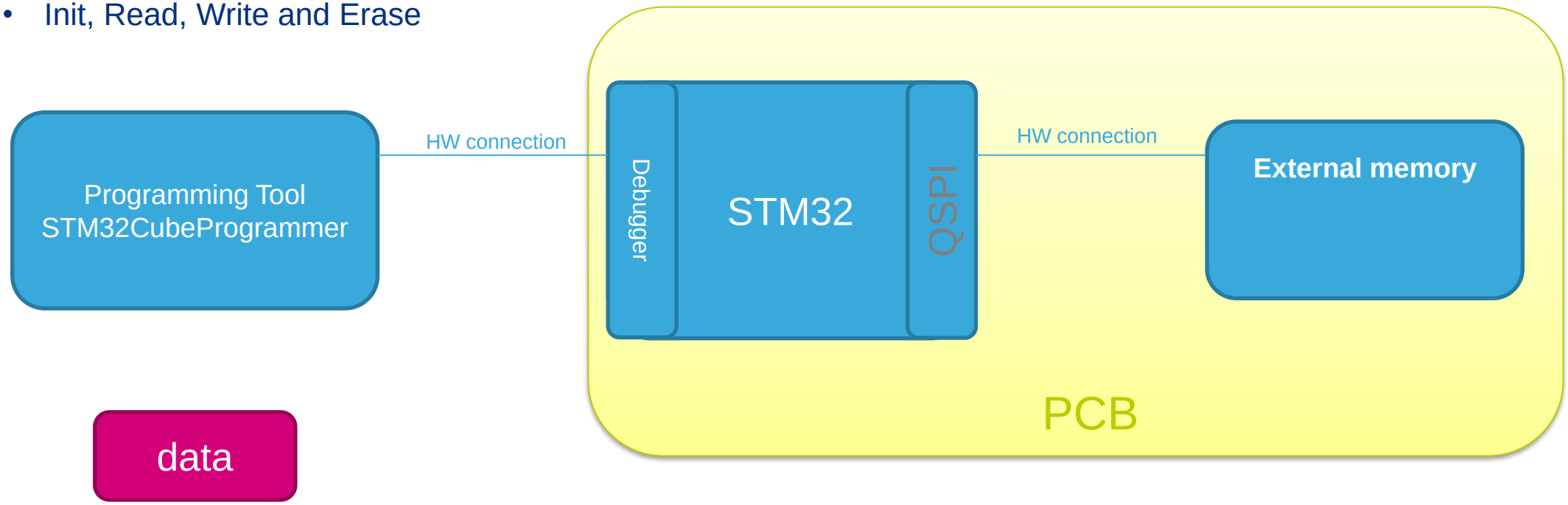
- Theory
  - What is External loader?
  - QSPI peripheral overview
  - Memory commands definition
- Hands-on
  - Create and setup project in CubeMX perspective
  - Validate QSPI functionality in basic project running from main flash
  - Modify project to external loader

Prerequisites

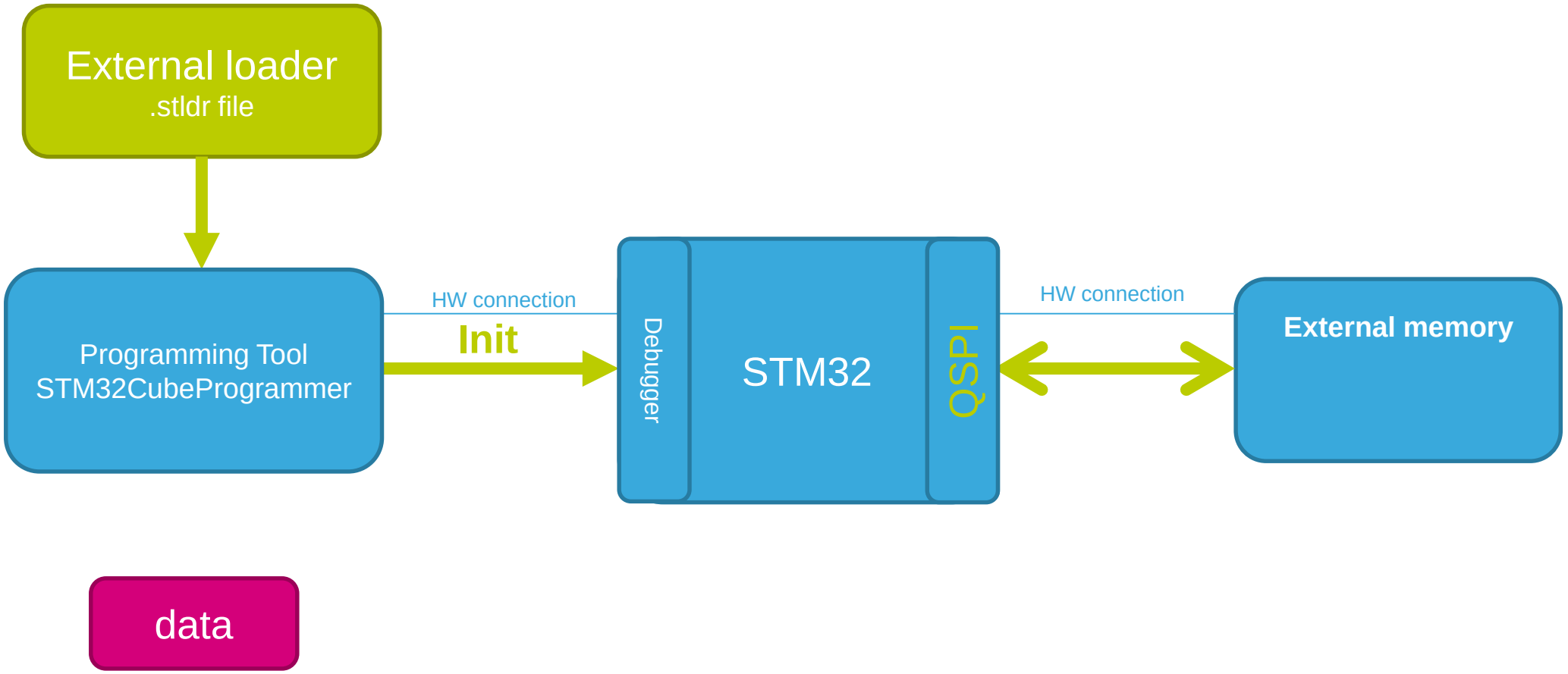
- STM32CubeIDE
- STM32CubeProgrammer
- Hardware - STM32F769I Discovery board
- Memory datasheet - MX25L51245G (MACRONIX)

External loader

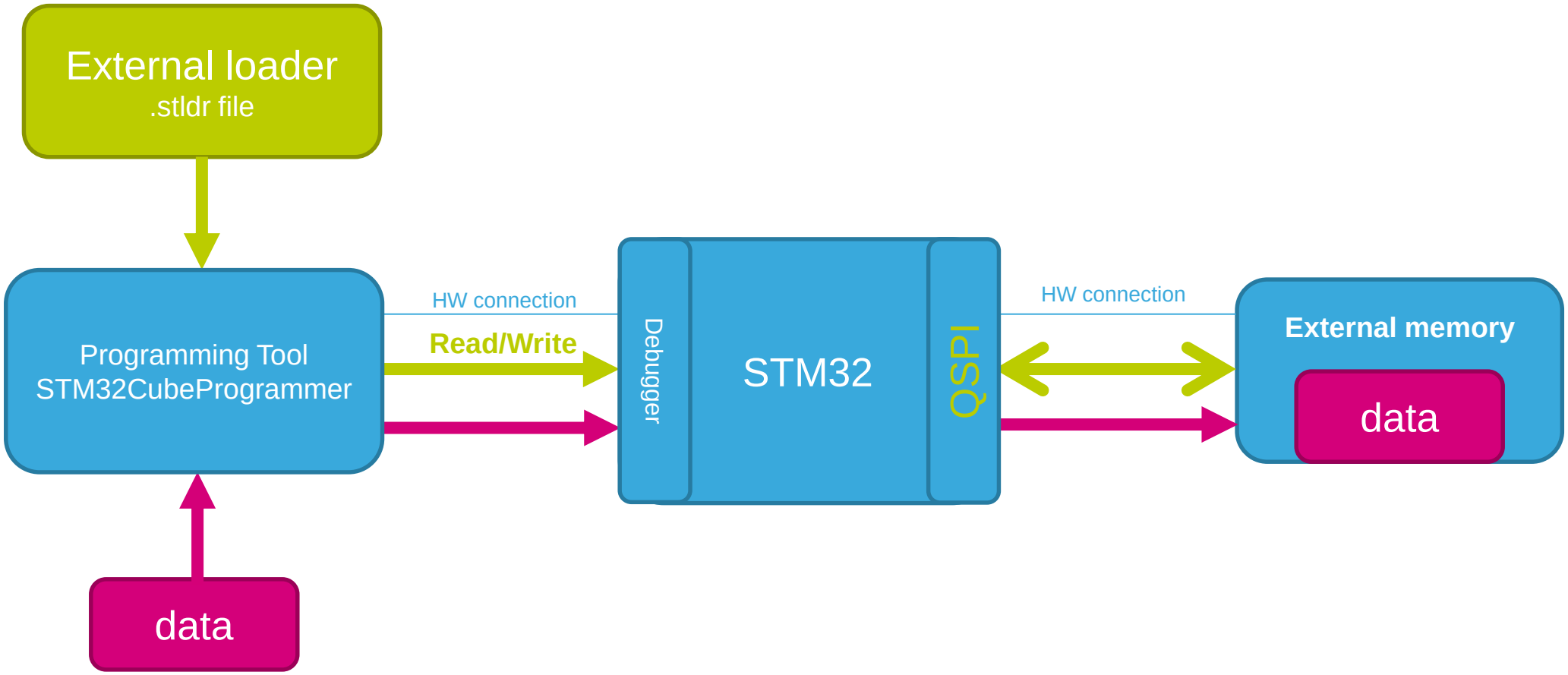
- Piece of code which is used for transfer data from PC to external memory through STM32
- .stldr file which contains memory functionality
  - Init, Read, Write and Erase



External loader



External loader



QSPI – general description

- Quad SPI communication interface – 6 pins
  - 4 data,
  - CLK
  - Chip Select
- Connect external memories
  - Data storage
  - Code execution
- Growing amount of manufacturers
- More complete QSPI theory is covered in AN4760

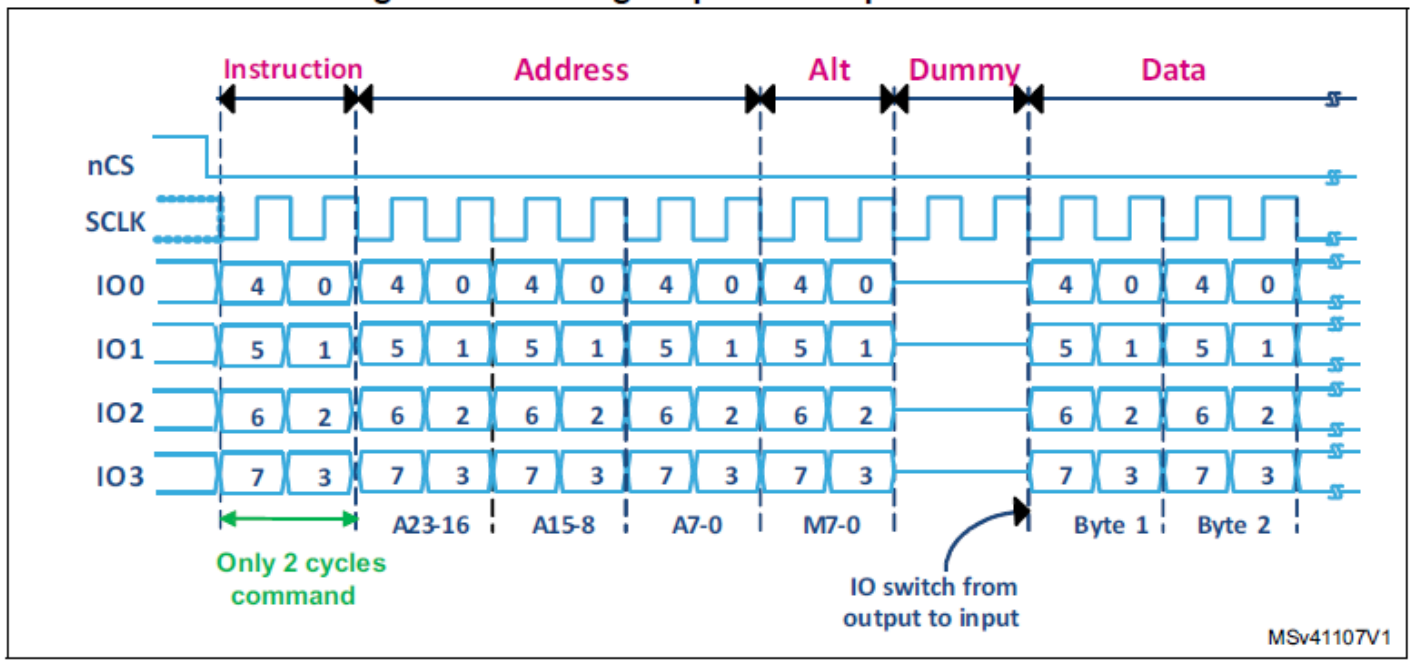
Advantages

- Better throughput than classical SPI
  - The classical SPI uses only one data line while the Quad-SPI uses four data lines which multiplies the data throughput by almost four times.
- Compared to FMC
  - allows to connect a **lower cost** external Flash memory
  - **Smaller packages**
  - **Lower pin** count
  - **Reducing PCB** area (easier design)

QSPI timing

- Example of QSPI timing sequence – each phase is configurable

Figure 5. Reading sequence in quad I/O SDR



QSPI modes

- Indirect mode
  - For reading, writing or erasing operations
  - If there is no need for AHB masters to access autonomously the QSPI memory (available in memory-mapped mode)
  - For all the operations to be performed through the Quad-SPI data registers using CPU or using DMA
- Status-flag mode
  - To read QSPI Flash memory status register
  - To poll autonomously for the end of an operation: Quad-SPI polls the status register inside the memory
- Memory-mapped mode
  - For reading operations
  - To use external QSPI Flash memory like an internal memory, so any AHB master can read data autonomously
  - For code execution from external QSPI Flash memory.



Example

STM32769I Discovery board

life.augmented

Workflow

Application test

Generated STM32 project

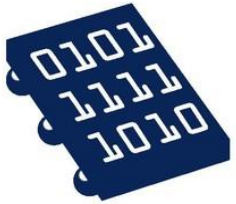




Specific loader files



- Dev_Inf.c/.h
- linker.ld
- Loader_Src.c



- quadspi.c
- quadspi.h

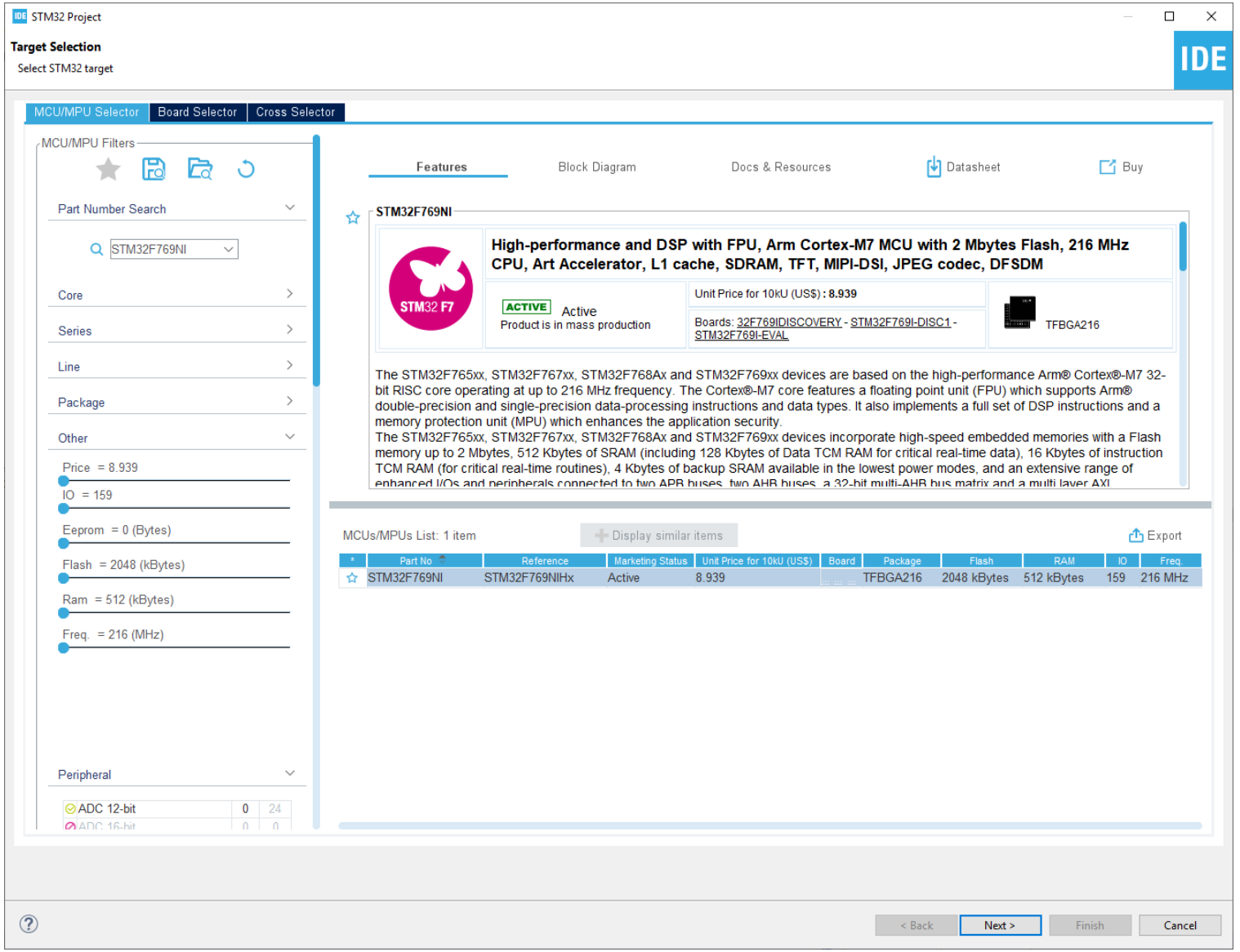
Memory specific functionality

Public storage

- **Memory 1**
  quadspi.c
  quadspi.h
- **Memory 2**
  quadspi.c
  quadspi.h

CubeIDE new project

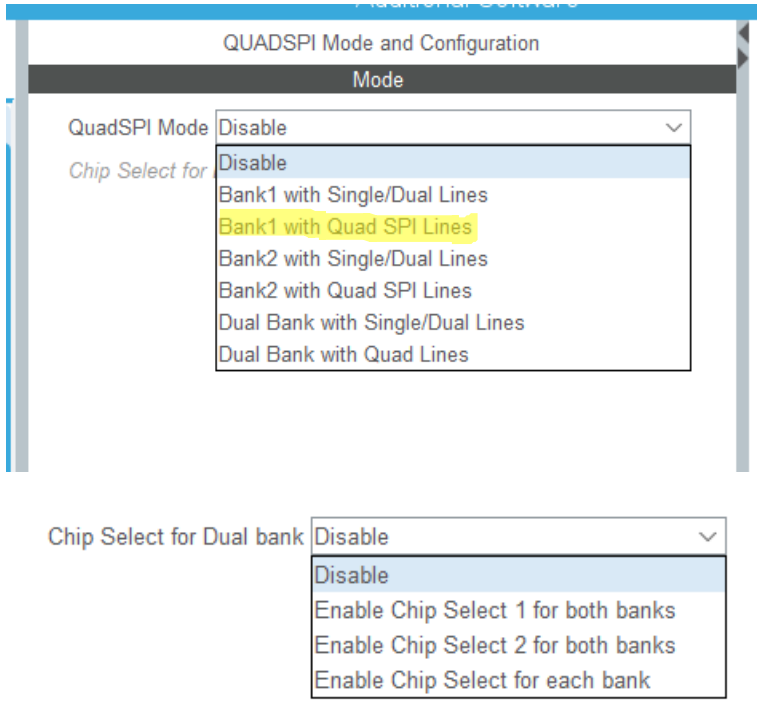
- Create project in CubeIDE
  - File > New > STM32 Project
  - Select STM32F769NI
  - Fill Project Name
  - Set location



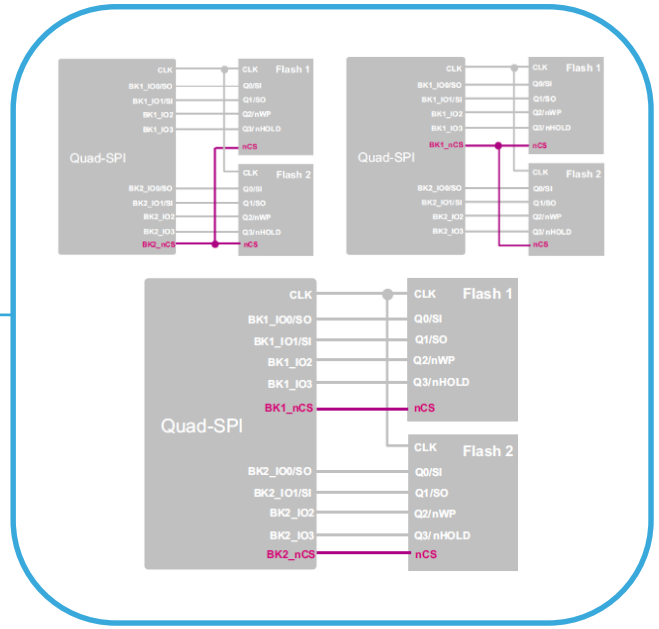
life.augmented

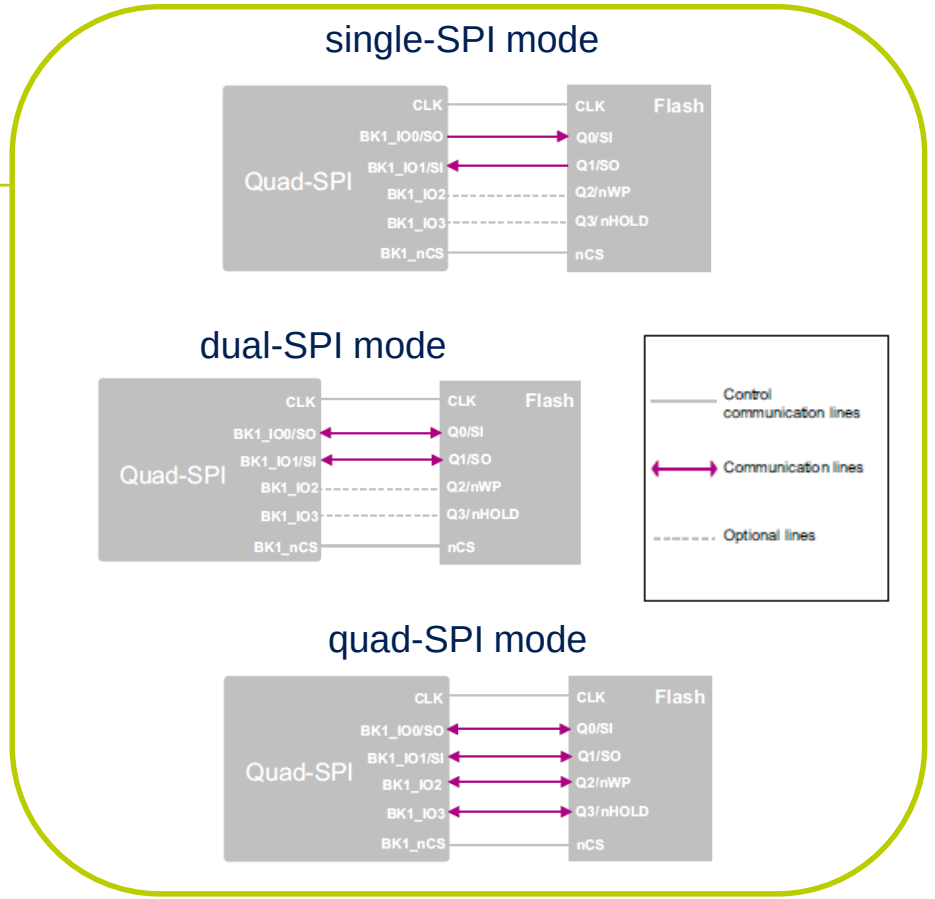
QSPI configuration

- STM32 supports Single, Dual and Quad mode for Bank1 and Bank2 (two flash memories)
  - Chip Select signals are fully configurable
  - One CS for both banks support usage of two memories at the same time (Dual-Flash memory mode)
    - Throughput and size are doubled - even addresses are stored in Flash 1, odd in Flash 2



* CS window in case of dual bank

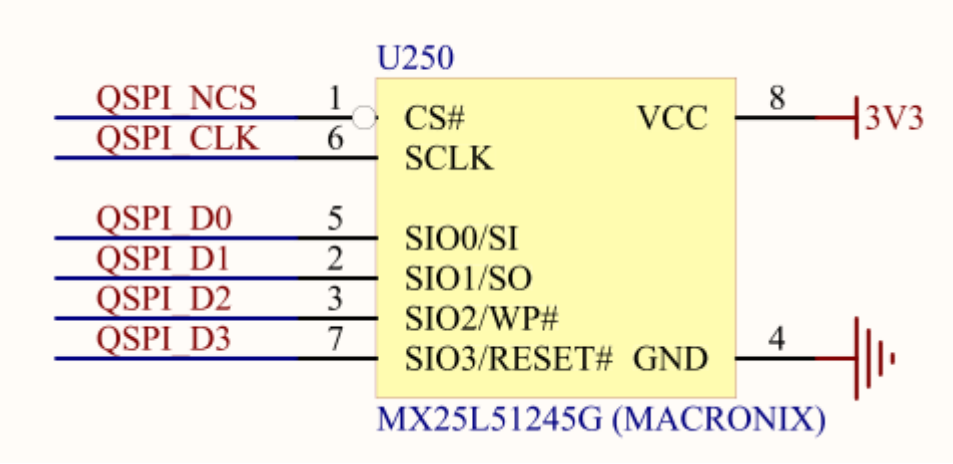


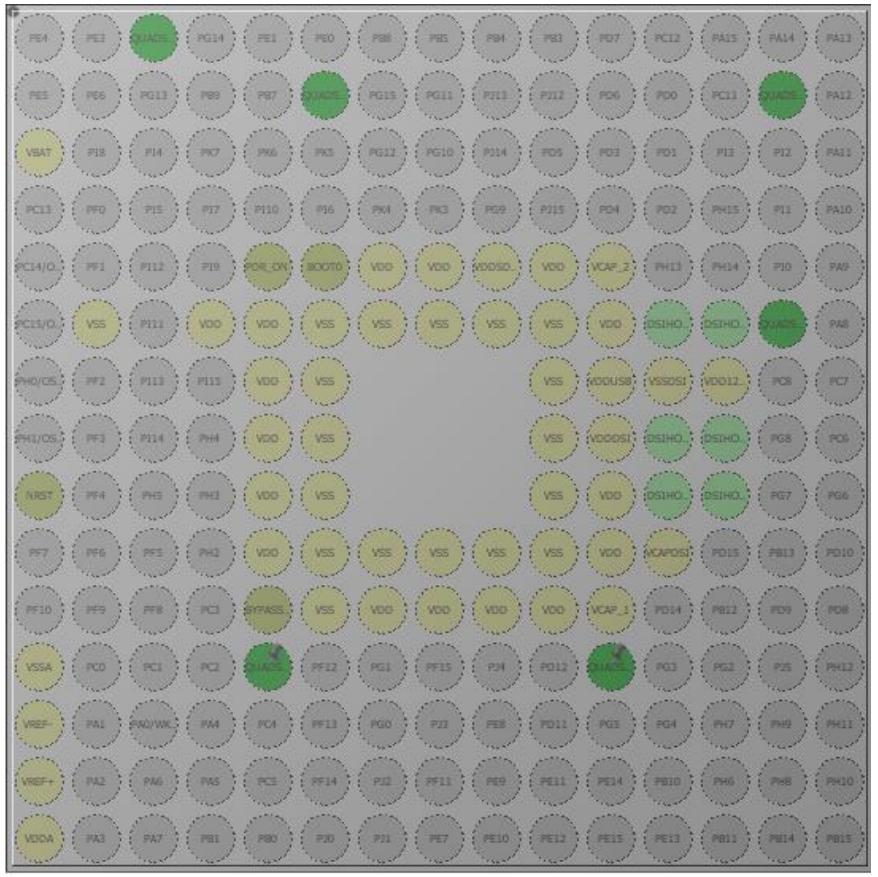


Pin assignment

- Check Pin assignment – compare with discovery board datasheet
  - Only some pins can be remapped to different GPIO pin (CTRL + Left mouse on pin)

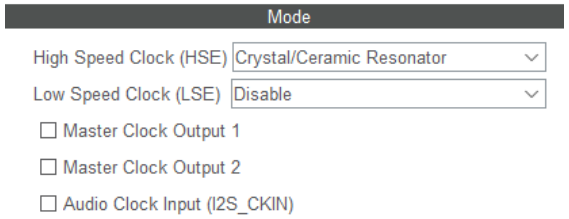
Pin Name	Signal on Pin
PB2	QUADSPI_CLK
PB6	QUADSPI_BK1_NCS
PC9	QUADSPI_BK1_IO0
PC10	QUADSPI_BK1_IO1
PD13	QUADSPI_BK1_IO3
PE2	QUADSPI_BK1_IO2



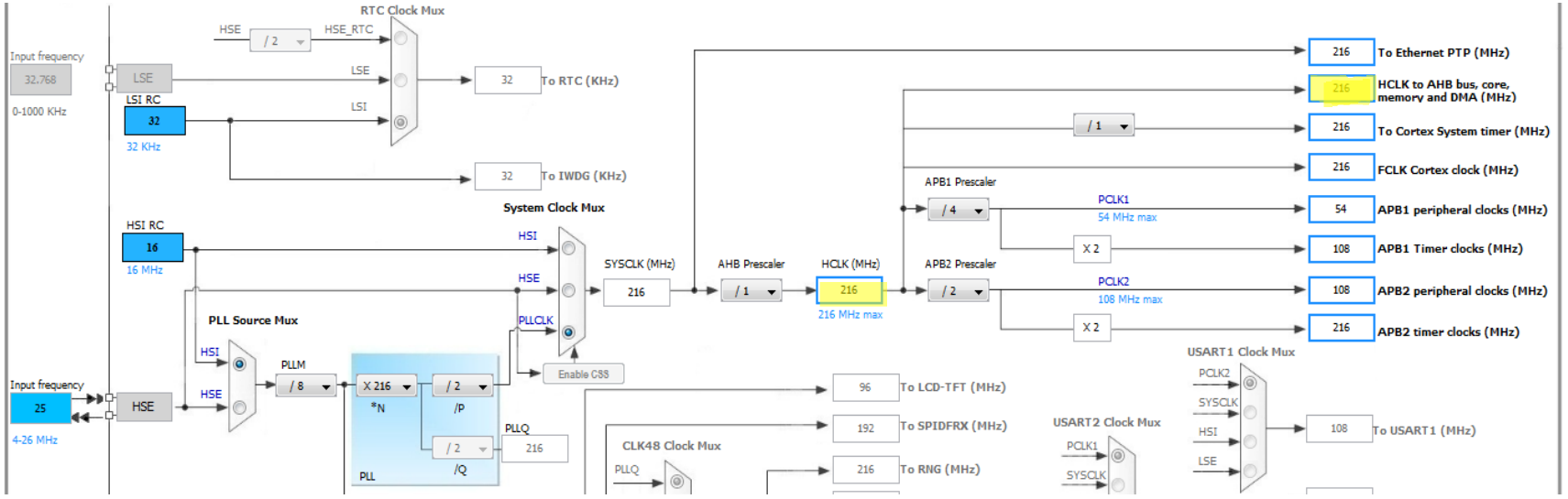


Clock configuration

- Select HSE clock source in RCC (if available, not mandatory)
  - Clock deviation need to be considerate when calculating memory timing

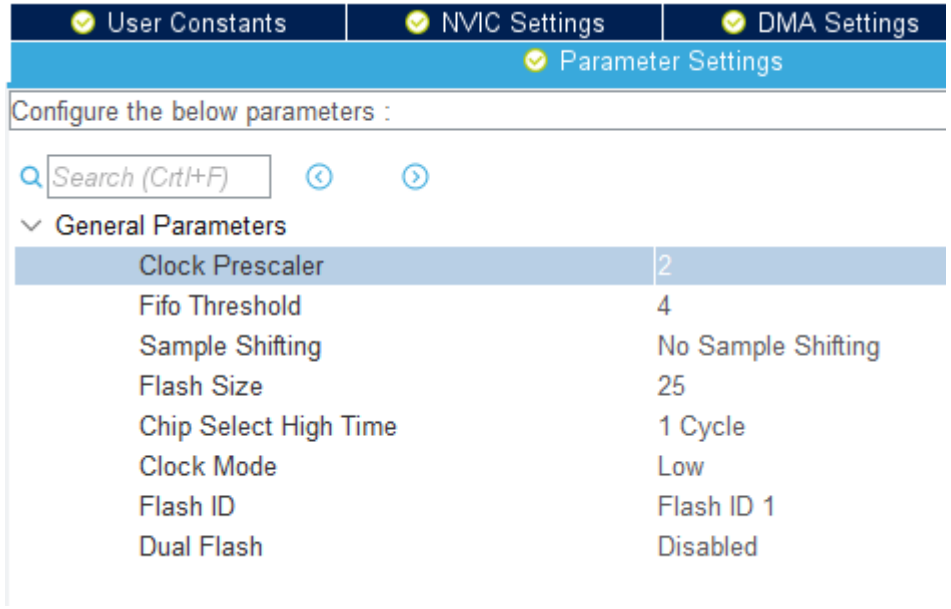


- Set your clock tree
  - AHB peripheral clock is used for QSPI peripheral



QSPI configuration

- Set QSPI
  - Set parameters from memory datasheet



QSPI clock = AHB clock / (ClockPrescaler + 1) = 72 MHz

FIFO Threshold = 4 => FTF flag is set as soon as there are five or more free bytes available to be written to the FIFO

Sample shifting half cycle disabled – may be needed with bad PCB design

Connected flash size – see equation on next slide

Chip select high time = 1 cycle

Data in setup time	$^{t}DVCH$	2	–	–	ns	
Data in hold time	$^{t}CHDX$	3	–	–	ns	
S# active hold time (relative to clock)	$^{t}CHSH$	4	–	–	ns	
S# not active setup time (relative to clock)	$^{t}SHCH$	4	–	–	ns	

life.augmented

QSPI configuration

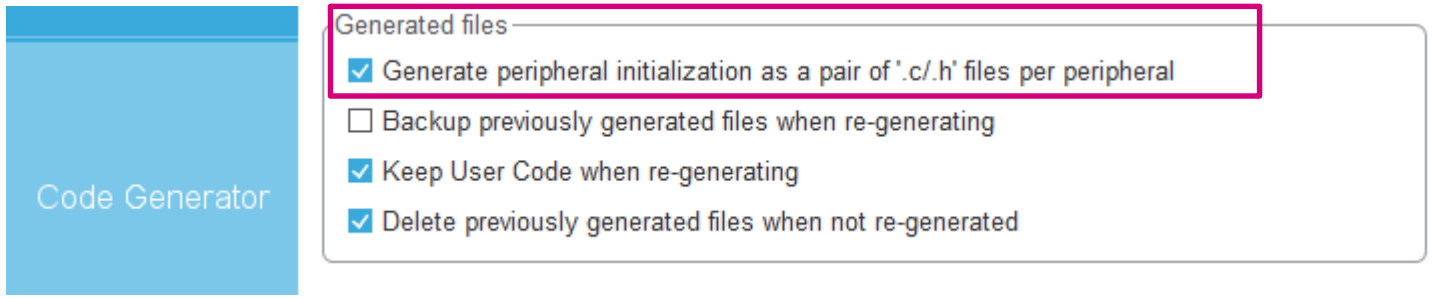
- Set QSPI
  - FSIZE calculation – set size of connected memory
  - Size is defined in memory chip datasheet

Size (Mbits)	Size (Mbytes)	Size (Bytes)	FSIZE
512	64	67108864	25
256	32	33554432	24
128	16	16777216	23
64	8	8388608	22
32	4	4194304	21

$$\textbf{FSIZE} = \frac{log(Size\ in\ bytes) - log2}{log2}$$

Generate a project

- Before generation we have to check
  "Generate peripheral initialization as a pair of .c/.c files per peripheral"



- Generate a code



QSPI commands

- Communication flow of most of the memories is standardized
  - Small deviation may be observed and need to be handled manually
    - For example 4 wire mode, number of dummy cycles settings, etc...
  - It may be hard during start, to orient in all available commands
- There are more parameters to be set for sending a command QSPI structure
  - Number of line for instruction
  - Number of bites and lines for address
  - Dummy cycles
  - Number of lines for data
- With higher number of lines, throughput is bigger

QSPI_CommandTypeDef

- All parameters should be filled
- Input values can be found in file *stm32f7xx_hal_qspi.h*
  - Try to find QSPI_DataMode as example

```
QSPI_CommandTypeDef sCommand;

sCommand.Instruction = ;          //value between 0x00 and 0xFF
sCommand.InstructionMode = ;      //defgroup QSPI_InstructionMode

sCommand.Address = ;              //value between 0x00 and 0xFFFFFFFF
sCommand.AddressSize = ;          //defgroup QSPI_AddressSize
sCommand.AddressMode = ;          //defgroup QSPI_AddressMode

sCommand.AlternateBytes = ;       //value between 0x00 and 0xFFFFFFFF
sCommand.AlternateBytesSize = ;   //defgroup QSPI_AlternateBytesSize
sCommand.AlternateByteMode = ;    //defgroup QSPI_AlternateBytesMode

sCommand.DummyCycles = ;          //number between 0 and 31

sCommand.DataMode = ;             //defgroup QSPI_DataMode
sCommand.NbData = ;               //value between 0x00 and 0xFFFFFFFF

sCommand.DdrMode = ;              //defgroup QSPI_DdrMode
sCommand.DdrHoldHalfCycle = ;     //defgroup QSPI_DdrHoldHalfCycle
sCommand.SIOOMode = ;             //defgroup QSPI_SIOOMode

HAL_QSPI_Command(&hqspi, &sCommand, HAL_QPSI_TIMEOUT_DEFAULT_VALUE);
```

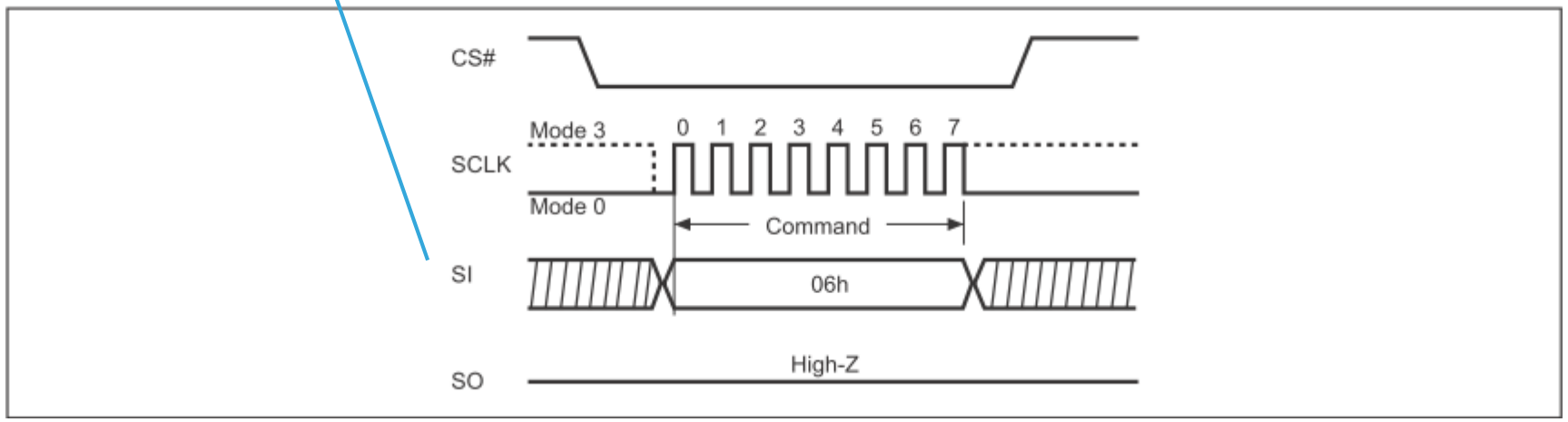
QSPI commands - example

- Write enable command
  - Command 1 line
  - No address line
  - No data line

```
QSPI_CommandTypeDef sCommand;

/* Enable write operations ---------------------------------------- */
sCommand.InstructionMode = QSPI_INSTRUCTION_1_LINE;
sCommand.Instruction = WRITE_ENABLE_CMD; //0x06
sCommand.AddressMode = QSPI_ADDRESS_NONE;
sCommand.AlternateByteMode = QSPI_ALTERNATE_BYTES_NONE;
sCommand.DataMode = QSPI_DATA_NONE;
sCommand.DummyCycles = 0;
sCommand.DdrMode = QSPI_DDR_MODE_DISABLE;
sCommand.DdrHoldHalfCycle = QSPI_DDR_HHC_ANALOG_DELAY;
sCommand.SIOOMode = QSPI_SIOO_INST_EVERY_CMD;
```

Figure 11. Write Enable (WREN) Sequence (SPI Mode)

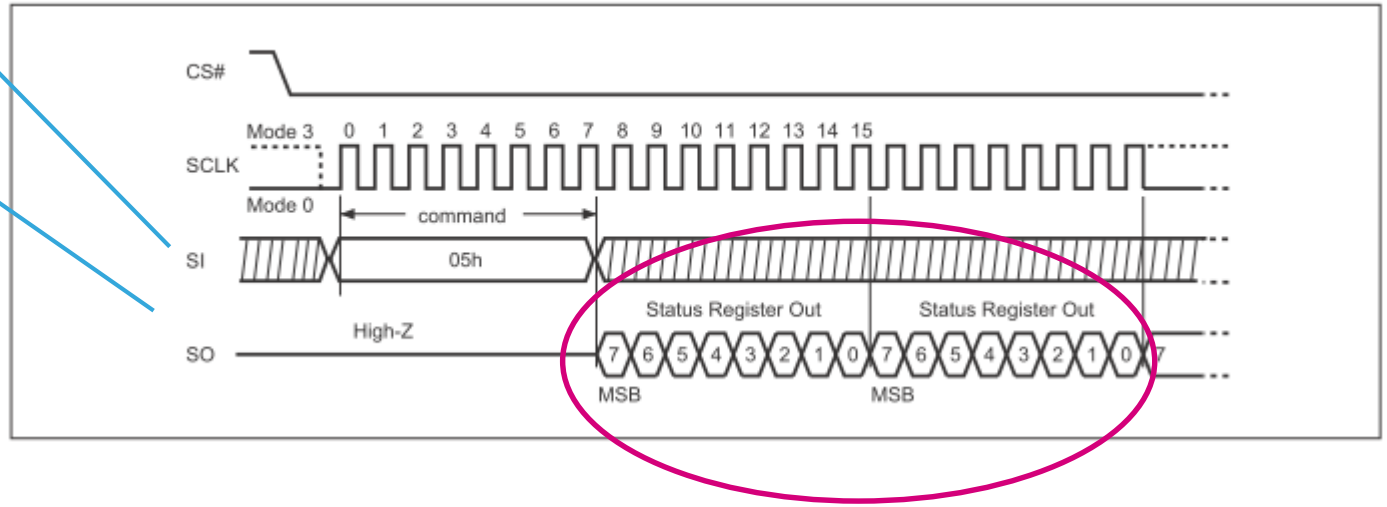


life.augmented

QSPI commands - example

- Read status register command
  - Command 1 line
  - No address line
  - Data 1 line
  - 2 bytes of data

Figure 21. Read Status Register (RDSR) Sequence (SPI Mode)

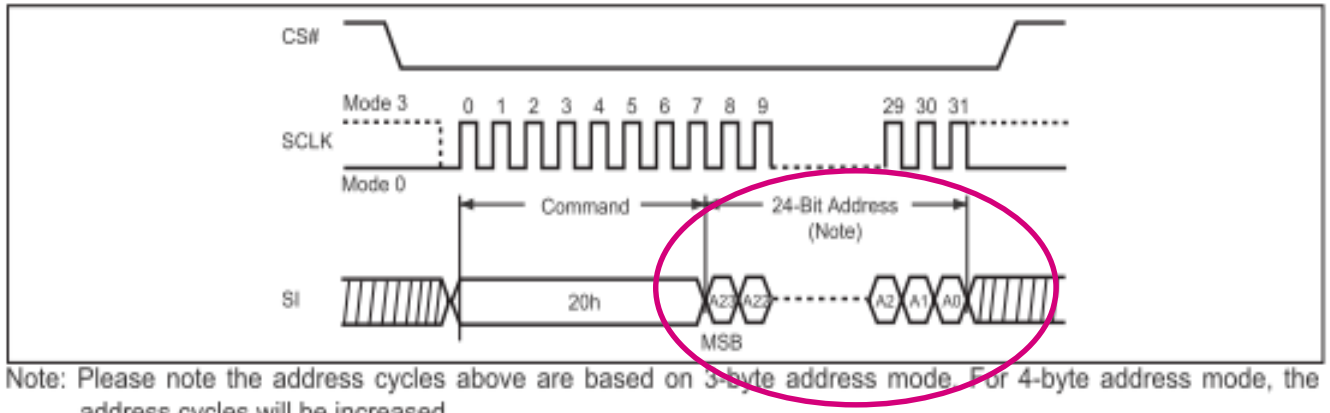


QSPI commands - example

- Sector erase command SPI mode
  - Command 1 line
  - Address 1 line
  - Address size 24 bits
  - no data line

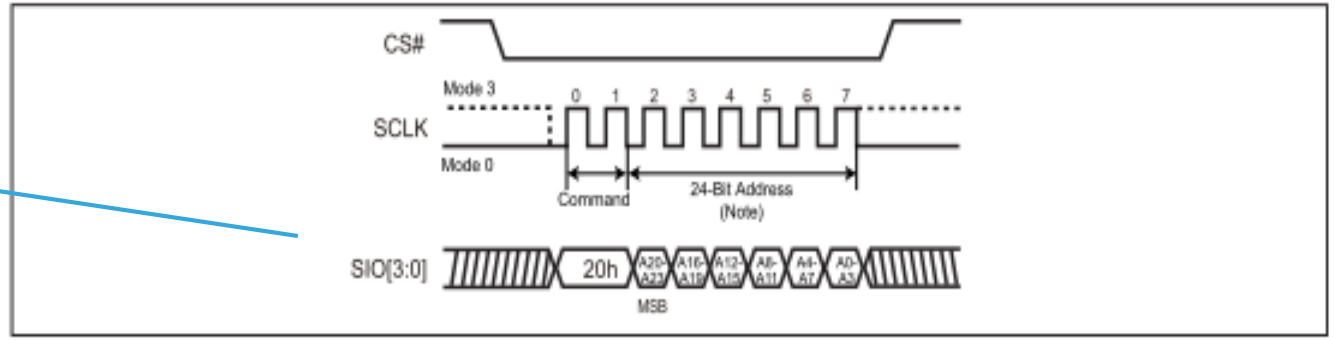
- Sector erase command QPI mode
  - Macronix chip specific
  - Switch to QPI mode needed before
  - Command 4 lines
  - Address 4 lines
  - Address size 24 bits
  - no data line

Figure 80. Sector Erase (SE) Sequence (SPI Mode)



Note: Please note the address cycles above are based on 3-byte address mode. For 4-byte address mode, the address cycles will be increased.

Figure 81. Sector Erase (SE) Sequence (QPI Mode)

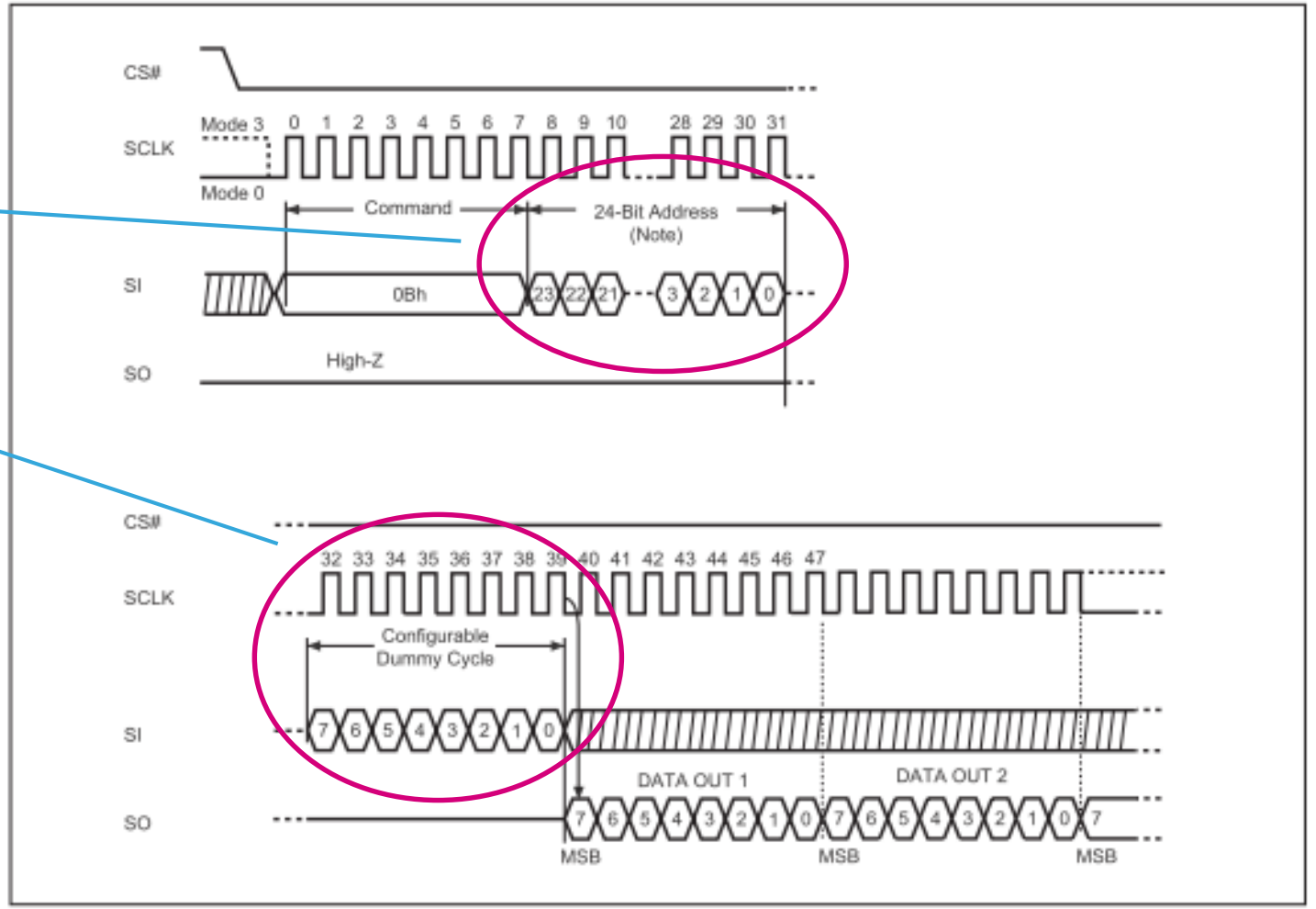


life.augmented

QSPI commands - example

- Read data bytes fast command
  - Command 1 line
  - Address 1 line
  - Address size 24 bits
  - X dummy bytes (frequency dependent)
  - Data 1 line
  - x bytes of data

Figure 32. Read at Higher Speed (FAST_READ) Sequence (SPI Mode)

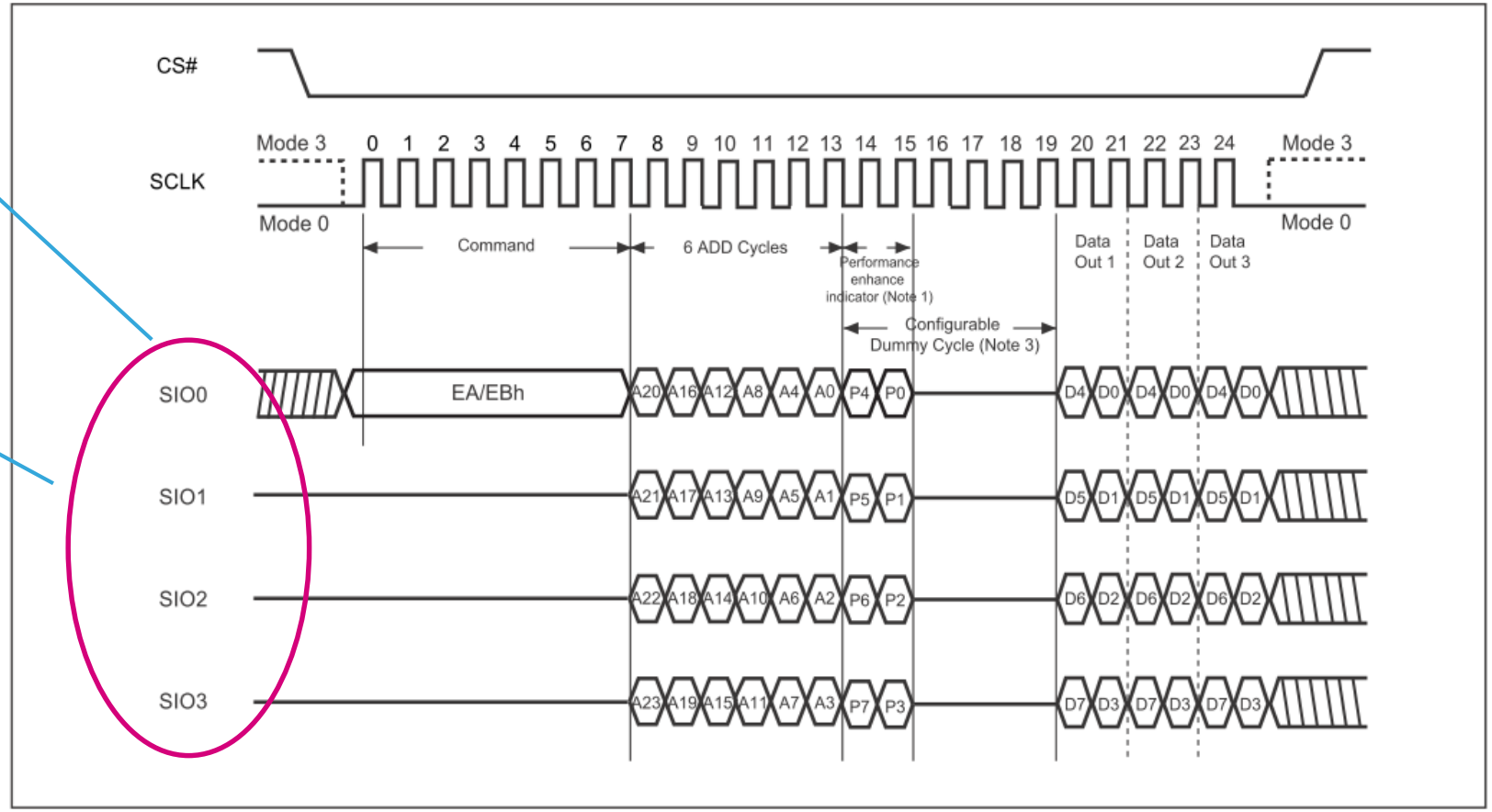


life.augmented

QSPI commands - example

- 4 IO read data bytes fast command
  - Command 1 line
  - Address 4 line
  - Address size 24 bits
  - X dummy bytes (frequency dependent)
  - Data 4 line
  - x bytes of data

Figure 36. 4 x I/O Read Mode Sequence (SPI Mode)

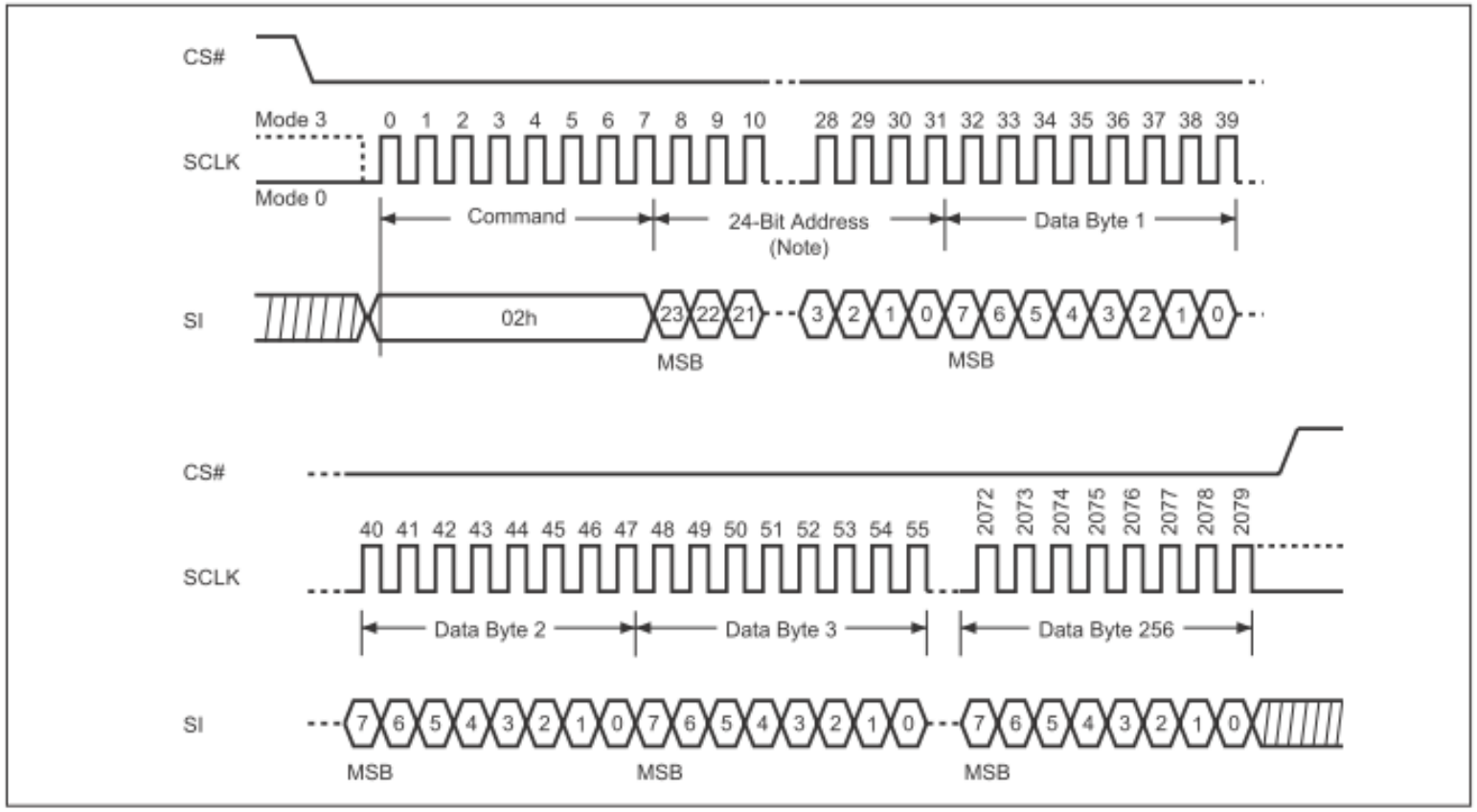


life.augmented

QSPI commands - example

- Write data bytes command
  - Command 1 line
  - Address 1 line
  - Address size 24 bits
  - 0 dummy bytes
  - Data 1 line
  - x bytes of data

Figure 88. Page Program (PP) Sequence (SPI Mode)

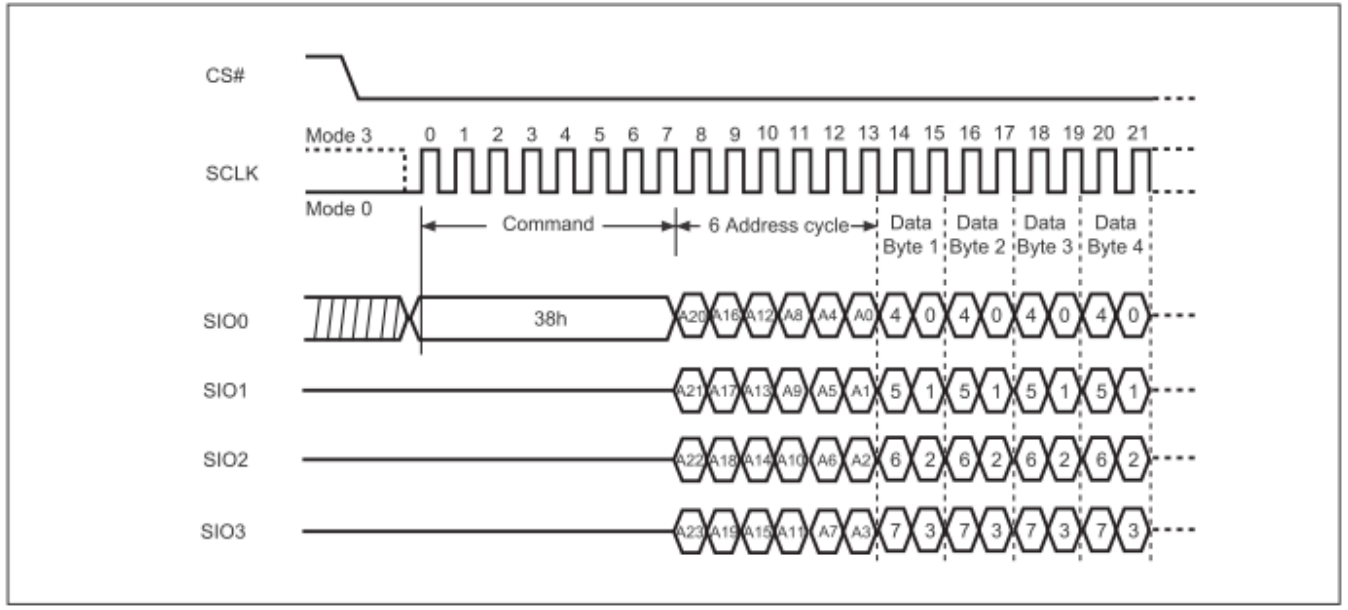


life.augmented

QSPI commands - example

- 4 IO Write data bytes command
  - Command 1 line
  - Address 4 line
  - Address size 24 bits
  - 0 dummy bytes (only for read)
  - Data 4 line
  - x bytes of data

Figure 90. 4 x I/O Page Program (4PP) Sequence (SPI Mode only)

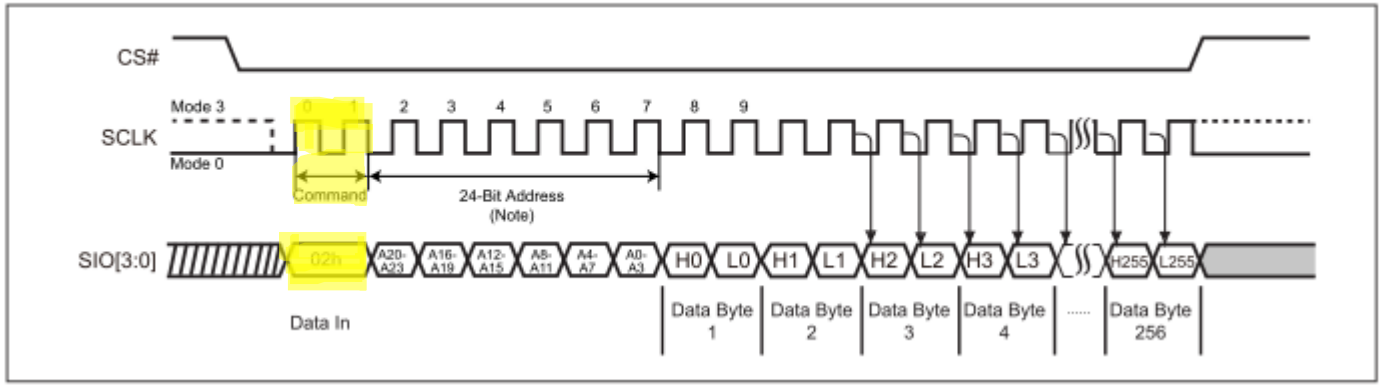


- Slightly slower compared to QPI write (8 cycles for command instead of 2), but no setting of mode is needed

QSPI commands - example

- Write data bytes command - QPI
  - Command 4 line
  - Address 4 line
  - Address size 24 bits
  - 0 dummy bytes (only for read)
  - Data 4 line
  - x bytes of data

Figure 89. Page Program (PP) Sequence (QPI Mode)



- Fastest option for write data into memory flash, but QPI mode need to be set before



QSPI_AutoPollingMemReady()

- Waiting for the memory to get ready for next operation
  - Used after erase/write operations
- sConfig Match and Mask must be aligned with memory datasheet

Status Register

bit2	bit1	bit0
BP0 (level of protected block)	WEL (write enable latch)	WIP (write in progress bit)
(note 1)	1=write enable 0=not write enable	1=write operation 0=not in write operation
Non-volatile bit	volatile bit	volatile bit

```
uint8_t QSPI_AutoPollingMemReady(void) {
QSPI_CommandTypeDef sCommand;
QSPI_AutoPollingTypeDef sConfig;
/* Configure automatic polling mode to wait for memory ready ------
*/
sCommand.InstructionMode = QSPI_INSTRUCTION_1_LINE;
sCommand.Instruction = READ_STATUS_REG_CMD;
sCommand.AddressMode = QSPI_ADDRESS_NONE;
sCommand.AlternateByteMode = QSPI_ALTERNATE_BYTES_NONE;
sCommand.DataMode = QSPI_DATA_1_LINE;
sCommand.DummyCycles = 0;
sCommand.DdrMode = QSPI_DDR_MODE_DISABLE;
sCommand.DdrHoldHalfCycle = QSPI_DDR_HHC_ANALOG_DELAY;
sCommand.SIOOMode = QSPI_SIOO_INST_EVERY_CMD;

sConfig.Match = 0x00;
sConfig.Mask = 0x01;
sConfig.MatchMode = QSPI_MATCH_MODE_AND;
sConfig.StatusBytesSize = 1;
sConfig.Interval = 0x10;
sConfig.AutomaticStop = QSPI_AUTOMATIC_STOP_ENABLE;

if (HAL_QSPI_AutoPolling(&hqspi, &sCommand, &sConfig,
HAL_QPSI_TIMEOUT_DEFAULT_VALUE) != HAL_OK) {
return HAL_ERROR;
}

return HAL_OK;
}
```

life.augmented

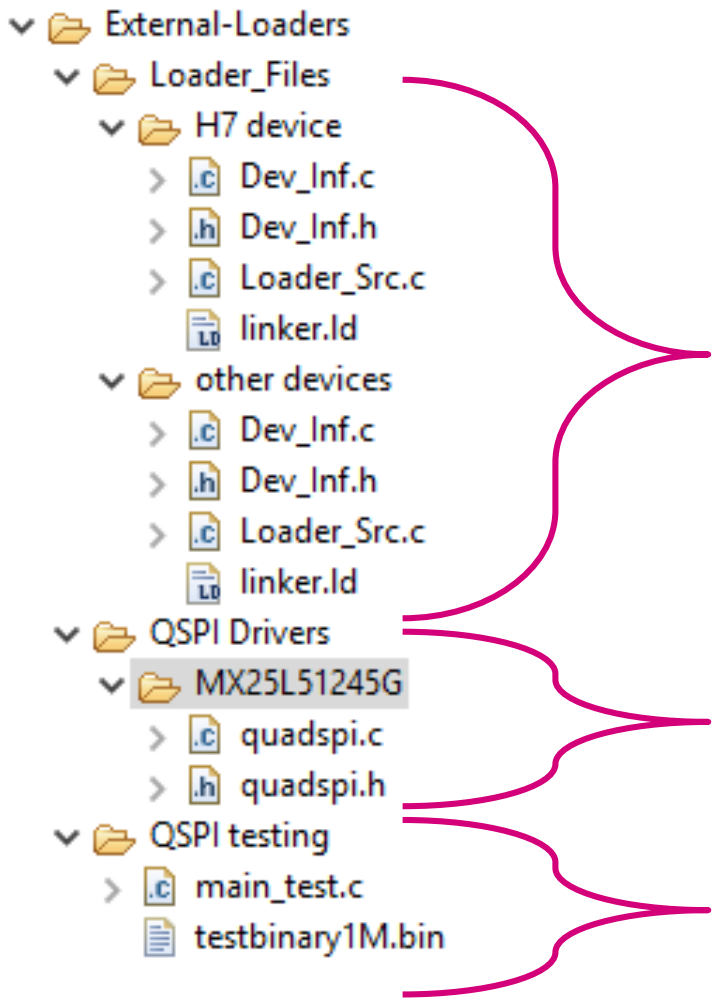
quadspi.c/.h files

- Based on CSP – chip support package
- Driver must contain functionality for
  - Initialization, Erase Sector, Full chip erase, Write and Enable Memory mapped mode
- .h file should defined sizes of memory

```
#define MEMORY_FLASH_SIZE    0x4000000   /* 512 MBits => 64MBytes */
#define MEMORY_BLOCK_SIZE    0x10000     /* 1024 sectors of 64KBytes */
#define MEMORY_SECTOR_SIZE   0x1000      /* 16384 subsectors of 4kBytes */
#define MEMORY_PAGE_SIZE     0x100       /* 262144 pages of 256 bytes */


uint8_t CSP_QUADSPI_Init(void);
uint8_t CSP_QSPI_EraseSector(uint32_t EraseStartAddress ,uint32_t EraseEndAddress);
uint8_t CSP_QSPI_WriteMemory(uint8_t* buffer, uint32_t address, uint32_t buffer_size);
uint8_t CSP_QSPI_EnableMemoryMappedMode(void);
uint8_t CSP_QSPI_Erase_Chip (void);
```

Public Storage



Loader and linker files for H7 or other devices (depends on STM32 type)

Memory drivers (growing)

QSPI testing routine + test 1MB binary

Memory specific functionality

INIT QSPI
CSP_QUAD_Init()

MX_QUADSPI_Init()
QSPI_ResetChip()
QSPI_AutoPollingMemReady()
QSPI_WriteEnable()
QSPI_Configuration()

Erase memory
CSP_QSPI_EraseSector()

QSPI_WriteEnable()
QSPI_AutoPollingMemReady()

Write buffer
CSP_QSPI_WriteMemory()

QSPI_WriteEnable()
QSPI_EnableMemoryMappedMode()
QSPI_AutoPollingMemReady()

Read buffer
- CSP_QSPI_EnableMemoryMappedMode()

main_test.c

```
/* USER CODE BEGIN 0 */
.
. Test implementation
.
/* USER CODE END 0 */


/* USER CODE BEGIN 2 */
.
. Test implementation
.
/* USER CODE END 2 */
```



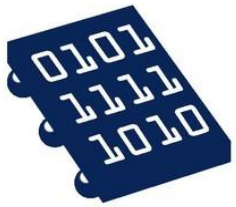
life.augmented

Workflow

Tested ✓

STM32 CubeIDE





- quadspi.c
- quadspi.h

Specific loader files



- Dev_Inf.c/.h
- linker.ld
- Loader_Src.c